

External Project Report on Computer Organisation And Architecture (EET 2211)

Digital Clock Using 8086 Assembly Language



Submitted by

Name 01 Pradeep Palai

Reg. No.: 2341016032

Name 02 Gopalaxmi Mohanty

Reg. No.: 2341013275

Name 03 Omkar Moharana

Reg. No.: 2341013397

Name 04 Ashutosh Nayak

Reg. No.: 2341014091

B. Tech. CSE 4th Semester (Section – M1)

**INSTITUTE OF TECHNICAL EDUCATION AND RESEARCH
(FACULTY OF ENGINEERING)**

**SIKSHA 'O' ANUSANDHAN (DEEMED TO BE UNIVERSITY), BHUBANESWAR,
ODISHA**

Declaration

We, the undersigned students of B. Tech. of **CSE** Department hereby declare that we own the full responsibility for the information, results etc. provided in this PROJECT titled “Digital Clock Using 8086 Assembly Language” submitted to **Siksha ‘O’ Anusandhan Deemed to be University, Bhubaneswar** for the partial fulfillment of the subject **COMPUTER ORGANISATION AND ARCHITECTURE (EET 1211)**. We have taken care in all respect to honor the intellectual property right and have acknowledged the contribution of others for using them in academic purpose and further declare that in case of any violation of intellectual property right or copyright we, as the candidate(s), will be fully responsible for the same.

PRADEEP PALAI

Registration No.: 2341016032

OMKAR MOHARANA

Registration No.:2341013397

GOPALAXMI MOHANTY

Registration No.: 2341013275

ASHUTOSH NAYAK

Registration No.:2341014091

DATE: 19/05/2025

PLACE: SOA, ITER

ABSTRACT

This project explores the development of a digital clock using the 8086 microprocessor, programmed entirely in assembly language. It aims to simulate a functional digital clock capable of displaying hours, minutes, and seconds in real-time using the GUI Turbo Assembler emulator. By manually simulating time progression through software delays, the project offers valuable insights into low-level system design and programming.

The primary challenge lies in the lack of access to real-time hardware within the GUI Turbo Assembler environment. To overcome this, the program uses nested loops to create accurate time delays that approximate a one-second interval. The clock display is managed by cursor positioning functions, ensuring that the screen updates occur at a fixed location.

This implementation also serves an educational purpose by providing students and enthusiasts with hands-on experience in using assembly instructions, memory management, BIOS interrupts, and arithmetic operations in 8086 architecture. Furthermore, the project strengthens the understanding of system timing and flow control, which are essential for embedded systems development.

Overall, this project is a stepping stone for more complex embedded systems projects and real-time applications, where accurate timing and resource-efficient code are paramount. It highlights the relevance of foundational computing knowledge in solving real-world problems through creative low-level programming techniques.

CONTENTS

1. Introduction
2. Objective
3. Tools and Technologies
4. System Overview
5. Methodology
6. Algorithm
7. Assembly Code
8. Output
9. Limitations
10. Future Enhancements
11. Conclusion

1.INTRODUCTION

The 8086 microprocessor, developed by Intel, represents a foundational step in the evolution of computer architecture and microcontroller-based applications. Despite its age, it remains a cornerstone in the study of low-level programming, digital system design, and embedded systems. In this project, we leverage the 8086's capabilities to design and implement a digital clock, which demonstrates timekeeping through simple arithmetic operations and BIOS interrupt-driven display routines.

The importance of this project stems from its relevance to the core concepts of embedded systems—especially where real-time data processing and precise control over system resources are required. With the increasing complexity of modern computing systems, it becomes even more critical for learners and developers to understand how low-level systems operate. By building a digital clock from scratch using assembly language, the programmer gains hands-on experience with direct hardware manipulation, interrupt usage, and time delay generation.

GUI Turbo Assembler (TASM) is used as the development environment due to its powerful macro support, assembler directives, and compatibility with x86 machine code. Since TASM does not provide direct support for real-time clock interfacing, our approach must rely on clever manipulation of loops and delays to mimic real-time behavior.

The choice of a clock as a project is deliberate—it is simple in concept but complex enough to explore major features of 8086 programming such as loop control, arithmetic operations, data display using interrupts, and program flow control. The project ensures that learners are exposed to a realistic problem where optimization, accuracy, and proper program structure are necessary to achieve a functional and user-friendly system.

Through this project, students can bridge the gap between theory and practical implementation, laying the groundwork for more advanced embedded system projects in the future. It not only tests their knowledge of instruction sets and memory addressing but also encourages a disciplined approach to software development in constrained environments.

2. OBJECTIVE

The objective of this project is to:

- Develop a simple digital clock that displays time in the format HH:MM:SS.
- Use basic arithmetic and I/O functions in assembly language.
- Simulate timing through delay loops due to the absence of a real-time clock in GUI Turbo Assembler.
- Display the time at a fixed screen position, updating every second.

3. TOOLS AND TECHNOLOGIES

Tool	Description
GUI Turbo	An x86 microprocessor emulator and assembler IDE.
Assembly Language	Low-level programming language for 8086 microprocessor.
DOS Interrupts	For input/output, cursor movement, and screen control.

4. SYSTEM OVERVIEW

Since GUI Turbo Assembler lacks access to the real-time clock hardware, we simulate time progression manually. The core of the program involves:

- Registers for holding hours, minutes, and seconds.
- A loop to increment time values.
- Cursor control using BIOS interrupts to keep the display in a single location.
- Delay routines to approximate real-time 1-second intervals

5. METHODOLOGY

The project follows a structured methodology encompassing planning, design, implementation, and testing phases:

1. Requirement Analysis: Identify the lack of real-time clock functionality in TASM and establish the need for delay loops to simulate time.
2. Program Design: Decide on the format (HH:MM:SS), storage registers for hour, minute, and second values, and routines for display.

3. Implementation:

Write routines to convert binary values to ASCII for display.

Develop delay routines using nested loops.

Create logic for incrementing seconds, handling rollovers to minutes and hours.

Use BIOS interrupts to move cursor and DOS interrupts to display characters.

4. Testing and Debugging: Run the program using TASM's integrated environment or with external DOSBox tools to validate the correctness of delays and time transitions.
5. Optimization: Fine-tune loop values and streamline code for efficient execution and readability.

This methodology ensures that the project adheres to good software engineering practices, even within the limitations of a low-level development environment.

6. ALGORITHM

1. Initialize data and code segments.
2. Set hours (hr), minutes (min), and seconds (sec) to zero.
3. Start an infinite loop:
 - Set cursor to top-left of screen.
 - Display HH:MM:SS using ASCII conversion and DOS output interrupts.
 - Delay for approximately 1 second using a nested loop.
 - Increment sec; if 60, reset to 0 and increment min.
 - If min becomes 60, reset to 0 and increment hr.
 - If hr becomes 24, reset to 0.

7. ASSEMBLY CODE

; Simple Digital Clock in 8086 Assembly
; Counts seconds properly with software-only implementation

.model small

.stack 100h

.data

; Time variables

hour db 12 ; Current hour (1-12)

minute db 0 ; Current minute (0-59)

second db 0 ; Current second (0-59)

is_pm db 0 ; 0 for AM, 1 for PM

; Display strings

msg db "Digital Clock (Press ESC to exit)", 0Dh, 0Ah, "\$"

time db "00:00:00 AM\$"

.code

main proc

mov ax, @data

mov ds, ax

; Clear screen

mov ax, 0600h ; Scroll entire window up (clear)

mov bh, 07h ; Normal attribute (white on black)

mov cx, 0000h ; Upper left corner (0,0)

mov dx, 184Fh ; Lower right corner (24,79)

int 10h

; Display message

mov ah, 09h

lea dx, msg

int 21h


```

; Initialize time from system
call init_time

; Main clock loop
clock_loop:
; Display current time
call display_time

; Delay for approximately 1 second
call delay

; Update time (increment second)
call update_time

; Check for ESC key
mov ah, 01h    ; Check if key pressed
int 16h
jz clock_loop  ; If no key, continue loop

mov ah, 00h    ; Get key
int 16h
cmp al, 27     ; Check if ESC (ASCII 27)
jne clock_loop ; If not ESC, continue loop

; Exit program
mov ax, 4C00h
int 21h
main endp

;-----
; Initialize time from system clock
;-----
init_time proc
; Get system time
mov ah, 2Ch    ; DOS function: Get System Time
int 21h
; CH = hour (0-23)
; CL = minute (0-59)

```

```

; DH = second (0-59)

; Convert 24-hour to 12-hour format
mov al, ch    ; AL = hour (0-23)

; Set AM/PM flag
cmp al, 12
jb am_time
mov is_pm, 1   ; PM
jmp hour_check
am_time:
mov is_pm, 0   ; AM

hour_check:
; Convert hour to 12-hour format
cmp al, 12    ; If hour = 12
je set_hour   ; 12 stays as 12
cmp al, 0     ; If hour = 0 (midnight)
je set_to_12  ; 0 becomes 12
cmp al, 12    ; If hour > 12
ja subtract_12 ; Subtract 12
jmp set_hour  ; Otherwise use as is

set_to_12:
mov al, 12    ; Set hour to 12
jmp set_hour

subtract_12:
sub al, 12    ; Convert to 12-hour format

set_hour:
mov hour, al  ; Store hour
mov minute, cl ; Store minute
mov second, dh ; Store second
ret
init_time endp
;-----

```

```

; Display current time
;-----
display_time proc
    ; Format time string
    call format_time

    ; Position cursor
    mov ah, 02h    ; Set cursor position
    mov bh, 0      ; Page 0
    mov dh, 2      ; Row 2
    mov dl, 0      ; Column 0
    int 10h

    ; Display time
    mov ah, 09h
    lea dx, time
    int 21h
    ret
display_time endp
;-----
; Format time string with current values
;-----
format_time proc
    ; Format hours
    mov al, hour
    call format_digit
    mov time[0], ah    ; Tens digit
    mov time[1], al    ; Ones digit

    ; Format minutes
    mov al, minute
    call format_digit
    mov time[3], ah    ; Tens digit
    mov time[4], al    ; Ones digit

    ; Format seconds
    mov al, second

```

```

    call format_digit
    mov time[6], ah    ; Tens digit
    mov time[7], al    ; Ones digit

; Set AM/PM
    cmp is_pm, 0
    je set_am
    mov time[9], 'P'   ; PM
    jmp format_done
set_am:
    mov time[9], 'A'   ; AM
format_done:
    ret
format_time endp

;-----
; Format a number (0-99) to ASCII digits
; Input: AL = number
; Output: AH = tens digit, AL = ones digit
;-----
format_digit proc
    mov ah, 0          ; Clear AH
    mov bl, 10          ; Divisor
    div bl              ; AL = quotient (tens), AH = remainder (ones)

    add al, '0'         ; Convert tens to ASCII
    xchg ah, al          ; Swap AH and AL
    add al, '0'         ; Convert ones to ASCII

    ret
format_digit endp

;-----
; Update time (increment second)
;-----
update_time proc
    ; Increment second
    inc second
    cmp second, 60

```

```

    jb done_update

    ; Second reached 60, reset and increment minute
    mov second, 0
    inc minute
    cmp minute, 60
    jb done_update

    ; Minute reached 60, reset and increment hour
    mov minute, 0
    inc hour

    ; Check if hour needs to roll over
    cmp hour, 12
    jbe check_noon
    mov hour, 1    ; Hour > 12, reset to 1
    jmp done_update

check_noon:
    ; Toggle AM/PM at noon or midnight
    cmp hour, 12
    jne done_update
    xor is_pm, 1    ; Toggle AM/PM
done_update:
    ret
update_time endp

;-----
; Simple software delay (approximately 1 second)
;-----

delay proc
    push ax
    push cx
    push dx

    ; Simple nested loop delay
    mov cx, 10    ; Outer loop count
outer_delay:

```

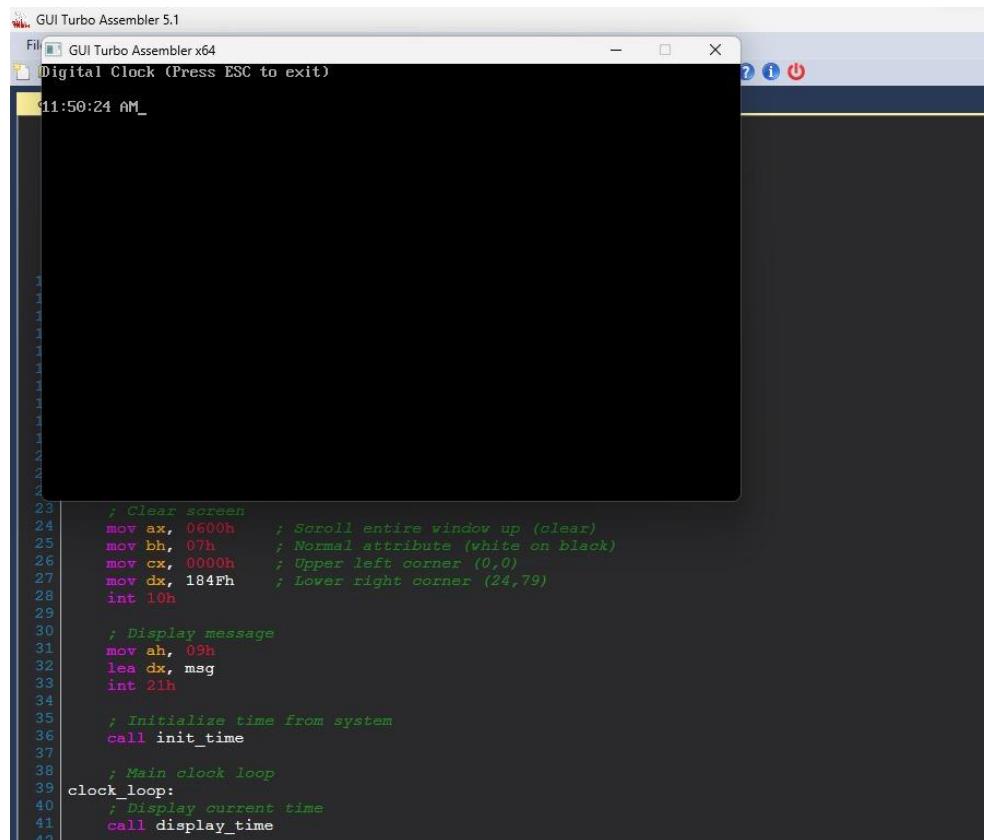
```
    push cx
    mov cx, 0FFFFh ; Inner loop count
inner_delay:
    nop            ; No operation (takes CPU cycles)
    nop            ; Additional NOPs to slow down
    nop
    nop
    loop inner_delay ; Decrement CX and loop until zero
    pop cx
    loop outer_delay ; Decrement CX and loop until zero

    pop dx
    pop cx
    pop ax
    ret
delay endp

end main
```

8. OUTPUT

Once the program is run in GUI Turbo Assembler x64 (using the "Emulate" > "Run" option), the top-left corner of the screen begins displaying:



```
GUI Turbo Assembler 5.1
GUI Turbo Assembler x64
Digital Clock (Press ESC to exit)
11:50:24 AM_

23 ; Clear screen
24 mov ax, 0600h ; Scroll entire window up (clear)
25 mov bh, 07h ; Normal attribute (white on black)
26 mov cx, 0000h ; Upper left corner (0,0)
27 mov dx, 184Fh ; Lower right corner (24,79)
28 int 10h
29
30 ; Display message
31 mov ah, 09h
32 lea dx, msg
33 int 21h
34
35 ; Initialize time from system
36 call init_time
37
38 ; Main clock loop
39 clock_loop:
40 ; Display current time
41 call display_time
42
```

Each second, this value updates in place — just like a digital clock. The format remains HH:MM:SS, and the values roll over correctly after 59 seconds, 59 minutes, and 23 hours.

9. LIMITATIONS

- This clock doesn't sync with the real system time — it's only a simulation.
- Timing may vary depending on system performance.
- No keyboard or user input features.
- No AM/PM support (24-hour format only).

10. FUTURE ENHANCEMENTS

- Sync with system time using DOS interrupt 1Ah (if supported).
- Add real-time input for setting time.
- Display date in addition to time.
- Sound an alarm at a set time using the PC speaker.

11. CONCLUSION

In conclusion, the development of a digital clock using 8086 assembly language in GUI Turbo Assembler (TASM) demonstrates the power and precision of low-level programming. It reinforces fundamental concepts such as register manipulation, loop control, BIOS and DOS interrupt handling, and arithmetic operations. Despite limitations like the absence of a built-in real-time clock, the project successfully simulates time progression using delay loops, showcasing how timing can be controlled purely through software. This hands-on experience offers a solid foundation for further exploration into embedded systems, where timing, efficiency, and direct hardware control are critical.

Moreover, this project illustrates how a relatively simple concept—timekeeping—can be a meaningful educational tool. By building the system from scratch, the programmer gains insight into the inner workings of microprocessors and the challenges of writing optimized, functional assembly code. The discipline required to manage program flow, screen output, and simulated delays provides invaluable experience in system-level thinking. This project not only achieves its goal of displaying a functional digital clock but also prepares learners for more complex hardware and software integration projects in the future.